

实验二：基于 Vision Transformer 的 CIFAR-10 图像分类

朱立景 2025E8013282177

一、概述

本实验旨在使用 **Vision Transformer (ViT)** 完成 CIFAR-10 数据集的 10 类图像分类任务，并在测试集上达到 **90% 以上** 的分类准确率。

任务定义： 给定一张 32×32 的 RGB 彩色图像，模型输出其所属类别（飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车）。

数据集： CIFAR-10，包含 50,000 张训练图像与 10,000 张测试图像，共 10 个类别，每类 6,000 张（训练 5,000 + 测试 1,000）。

解决方案： 针对 CIFAR-10 的小分辨率特点，设计轻量级 ViT 网络（Patch Size = 4，Embed Dim = 192，Depth = 9），配合 RandAugment、Mixup/CutMix、Label Smoothing 等训练技巧，采用 AdamW 优化器与 Cosine 学习率调度，在 GPU 上训练 200 个 epoch。

实验环境： Conda, Python 3.10, PyTorch 2.5.1 + CUDA, NVIDIA A800 GPU。

实验结果： 最佳测试准确率 **91.11%**（第 183 epoch），达到预设目标（≥ 90%）。

二、解决方案

2.1 整体流程

1 | CIFAR-10 图像 → 数据增强 & 归一化 → Patch 切分 & 嵌入 → ViT Encoder → CLS Token → 线性分类头 → 10 类 logits

训练阶段对输入图像施加随机裁剪、翻转、RandAugment、Random Erasing 等增强，并以 Mixup / CutMix 混合样本；测试阶段仅做 ToTensor 与标准化。模型以 [CLS] token 的最终表示作为全局图像特征，经线性层输出类别概率。

2.2 网络结构设计

模型 `VisionTransformer` 遵循 Dosovitskiy 等人提出的 ViT 架构，针对 32×32 小图做了尺度适配：

full-size ViT 通常使用 16×16 patch，本实验采用 **4×4 patch**，使 32×32 图像产生 $8 \times 8 = 64$ 个 patch，保留足够的空间分辨率供 Transformer 建模。

模块	说明
Patch Embedding	4×4 卷积 (stride=4) 将图像切分并投影到 192 维
CLS Token	可学习分类 token，拼接于 patch 序列首位
Positional Embedding	可学习位置编码，长度 = 64 + 1
Transformer Encoder	9 层，每层含 Pre-Norm 多头自注意力 + MLP
分类头	LayerNorm + Linear(192 → 10)

超参数配置：

参数	取值
img_size	32
patch_size	4
embed_dim	192
depth	9
num_heads	6
mlp_ratio	4.0
drop_rate	0.1
参数量	4,028,170

Patch 嵌入与 ViT 前向传播核心代码：

```
1 class PatchEmbed(nn.Module):
2     def __init__(self, img_size=32, patch_size=4, in_chans=3, embed_dim=192):
3         super().__init__()
4         self.grid_size = img_size // patch_size
5         self.num_patches = self.grid_size ** 2
6         self.proj = nn.Conv2d(in_chans, embed_dim, kernel_size=patch_size, stride=patch_size)
7
8     def forward(self, x):
9         x = self.proj(x)
10        return x.flatten(2).transpose(1, 2) # (B, 64, 192)
11
12 class VisionTransformer(nn.Module):
13     def forward(self, x):
14         b = x.shape[0]
15         x = self.patch_embed(x)
16         cls = self.cls_token.expand(b, -1, -1)
17         x = torch.cat([cls, x], dim=1) # (B, 65, 192)
18         x = x + self.pos_embed
19         x = self.pos_drop(x)
20         x = self.blocks(x)
21         x = self.norm(x)
22         return self.head(x[:, 0]) # 取 CLS token 分类
```

Transformer Block 采用 Pre-Norm 结构（先 LayerNorm 再 Attention/MLP），残差连接如下：

```
1 class TransformerBlock(nn.Module):
2     def forward(self, x):
3         x = x + self.attn(self.norm1(x))
4         x = x + self.mlp(self.norm2(x))
5         return x
```

2.3 损失函数设计

训练时使用带 **Label Smoothing** 的交叉熵损失（`label_smoothing=0.1`），缓解过拟合并提升泛化。当启用 Mixup / CutMix 时，损失对两个混合标签加权求和：

$$\mathcal{L} = \lambda \cdot \text{CE}(\hat{y}, y_a) + (1 - \lambda) \cdot \text{CE}(\hat{y}, y_b)$$

其中 $\lambda \sim \text{Beta}(\alpha, \alpha)$, Mixup 取 $\alpha = 0.8$, CutMix 取 $\alpha = 1.0$, 二者以 50% 概率随机选用。

```
1 criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
2
3 def mixup_criterion(criterion, pred, y_a, y_b, lam):
4     return lam * criterion(pred, y_a) + (1.0 - lam) * criterion(pred, y_b)
```

评估阶段使用标准交叉熵（无 Label Smoothing、无 Mixup），以 argmax 准确率作为指标。

2.4 优化器与学习率调度

优化器：AdamW，基础学习率 `5e-4`，权重衰减 `0.05`。对 bias 与 LayerNorm 参数不做 weight decay，其余权重参数施加衰减，这是 ViT 训练的常见做法。

```
1 optimizer = optim.AdamW([
2     {"params": decay, "weight_decay": 0.05},
3     {"params": no_decay, "weight_decay": 0.0},
4 ], lr=5e-4, betas=(0.9, 0.999))
```

学习率调度：前 10 个 epoch 线性 Warmup，之后 Cosine 退火至 0：

```
1 def lr_lambda(epoch):
2     if epoch < warmup_epochs:
3         return (epoch + 1) / warmup_epochs
4     progress = (epoch - warmup_epochs) / (epochs - warmup_epochs)
5     return 0.5 * (1.0 + math.cos(math.pi * progress))
```

其他训练细节：

- Batch Size = 128
- 梯度裁剪： `max_norm = 1.0`
- 随机种子：42

2.5 数据增强策略

阶段	增强方法
训练	RandomCrop(32, padding=4)、RandomHorizontalFlip、RandAugment(2 ops, magnitude=9)、Normalize、RandomErasing(p=0.25)
测试	ToTensor + Normalize

归一化使用 CIFAR-10 数据集统计量：均值 `(0.4914, 0.4822, 0.4465)`，标准差 `(0.2470, 0.2435, 0.2616)`。

2.6 工程优化（创新点）

1. **小图 ViT 适配：**将 patch size 设为 4（而非 ImageNet 常用的 16），使 32×32 图像仍保留 64 个 patch，避免序列过短导致 ViT 无法有效建模空间关系。
2. **组合式正则化：**同时启用 Mixup、CutMix 与 Label Smoothing，三者协同抑制 ViT 在 CIFAR-10 小数据集上的过拟合倾向。
3. **国内镜像下载：**实现 `ensure_cifar10()`，优先从百度 BOS 镜像下载数据集，失败时自动回退官方源，并校验 MD5，提

升实验可复现性与下载效率。

三、实验分析

3.1 数据集介绍

CIFAR-10 是计算机视觉领域最经典的基准数据集之一，由 Hinton 等人收集整理：

属性	说明
图像尺寸	32 × 32 × 3 (RGB)
类别数	10
训练集	50,000 张
测试集	10,000 张
类别	airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

该数据集图像分辨率小、类内差异大，对模型的特征提取与泛化能力均有较高要求。ViT 原论文在 ImageNet 上以大规模预训练为主，直接在小数据集、小分辨率上训练 ViT 难度较大，因此本实验重点依赖数据增强与正则化策略。

3.2 实验配置

配置项	取值
设备	CUDA
Epochs	200
Batch Size	128
学习率	5×10^{-4}
Weight Decay	0.05
Warmup Epochs	10
Mixup α	0.8
CutMix α	1.0
Label Smoothing	0.1

3.3 实验结果

最终指标：

指标	结果
最佳测试准确率	91.11% (Epoch 183)
最终测试准确率	91.03% (Epoch 200)
最佳测试 Loss	0.3891 (Epoch 185)
目标准确率	≥ 90% ✓

训练过程关键节点：

Epoch	测试准确率	测试 Loss	说明
1	26.83%	1.9464	随机初始化，模型尚未收敛
10	53.84%	1.3322	Warmup 结束，学习率达到峰值
50	77.87%	0.7815	快速提升期
100	86.59%	0.5068	进入 85%+ 区间
135	89.78%	0.4242	接近 90%
149	90.06%	0.4185	首次突破 90%
156	90.62%	0.4170	稳定在 90% 以上
183	91.11%	0.3959	全局最优
200	91.03%	0.3946	训练结束，略有平台期

收敛曲线特征分析：

- 1. **Warmup 阶段 (Epoch 1-10)：** 学习率从 1×10^{-4} 线性升至 5×10^{-4} ，准确率由 26.83% 快速升至 53.84%，模型在学习基础特征。
- 2. **快速学习期 (Epoch 11-80)：** Cosine 调度下准确率持续攀升，Epoch 80 达到 84.86%，Loss 从 1.3+ 降至 0.59 左右。
- 3. **精细调优期 (Epoch 81-150)：** 增速放缓，Epoch 100-150 准确率从 86.59% 提升至 90.11%，突破 90% 门槛。
- 4. **平台收敛期 (Epoch 151-200)：** 准确率在 90%-91.1% 之间波动，Loss 稳定在 0.39-0.43，模型已充分收敛，继续训练收益有限。

Loss 与准确率关系： 训练 Loss 全程高于测试 Loss（例如 Epoch 200 训练 Loss 1.35 vs 测试 Loss 0.39），这是 Mixup/CutMix 与 Label Smoothing 的预期现象——训练目标被「软化」，而评估使用硬标签，二者不可直接对比，但测试准确率的持续上升表明正则化策略有效。

单 Epoch 耗时： 约 10-16 秒（GPU），200 epoch 总训练时间约 35-45 分钟。

四、总结

本实验成功实现了基于 Vision Transformer 的 CIFAR-10 图像分类系统。通过将 patch size 缩小至 4x4、控制模型规模（约 400 万参数），并配合 RandAugment、Mixup/CutMix、Label Smoothing 及 AdamW + Cosine 调度等训练策略，在 200 个 epoch 后于测试集取得 **91.11%** 的最佳准确率，超过 90% 的实验目标。

实验表明：ViT 虽为 Transformer 架构，在缺乏大规模预训练的情况下，只要针对小分辨率图像合理设计 patch 粒度，并辅以充分的数据增强与正则化，同样可以在 CIFAR-10 上取得与优秀 CNN 相当甚至更优的性能。后续可进一步探索的知识蒸馏（DeiT 思路）、更长的训练周期或 AutoAugment 策略，有望将准确率推向 92% 以上。